[1]
✓ 0s

```python
from typing import List, Dict, Any, Tuple

def _validate_input(reference: List[int], num_frames: int) -> None:
    if not reference:
        raise ValueError("The page reference string must contain at least one page.")
    if not isinstance(num_frames, int) or num_frames < 1:
        raise ValueError("Number of frames must be an integer greater than or equal to 1.")
    if any(not isinstance(page, int) for page in reference):
        raise ValueError("Every page reference must be an integer.")

def fifo(reference: List[int], num_frames: int) -> Tuple[int, int, List[Dict[str, Any]]]:
    """FIFO replaces the page that entered memory earliest."""
    _validate_input(reference, num_frames)
    frames = []
    pointer = 0
    trace = []
    faults = hits = 0
    for step, page in enumerate(reference, start=1):
        replaced = None
        if page in frames:
            hits += 1
            status = "Hit"
        else:
            faults += 1
            status = "Fault"
            if len(frames) < num_frames:
                frames.append(page)
            else:
                replaced = frames[pointer]
                frames[pointer] = page
                pointer = (pointer + 1) % num_frames
        trace.append({"step": step, "page": page, "frames": frames.copy(),
                      "status": status, "replaced": replaced})
    return faults, hits, trace
```

[1]
✓ 0s

```python
def lru(reference: List[int], num_frames: int) -> Tuple[int, int, List[Dict[str, Any]]]:
    """LRU replaces the page whose most recent use was farthest in the past."""
    _validate_input(reference, num_frames)
    frames = []
    last_used = {}
    trace = []
    faults = hits = 0
    for step, page in enumerate(reference, start=1):
        replaced = None
        if page in frames:
            hits += 1
            status = "Hit"
        else:
            faults += 1
            status = "Fault"
            if len(frames) < num_frames:
                frames.append(page)
            else:
                victim = min(frames, key=lambda p: last_used[p])
                replaced = victim
                frames[frames.index(victim)] = page
        last_used[page] = step
        trace.append({"step": step, "page": page, "frames": frames.copy(),
                      "status": status, "replaced": replaced})
    return faults, hits, trace
```

[1] ✓ 0s

```python
def optimal(reference: List[int], num_frames: int) -> Tuple[int, int, List[Dict[str, Any]]]:
    """Optimal replaces the page whose next use is farthest in the future."""
    _validate_input(reference, num_frames)
    frames = []
    trace = []
    faults = hits = 0
    for i, page in enumerate(reference):
        replaced = None
        if page in frames:
            hits += 1
            status = "Hit"
        else:
            faults += 1
            status = "Fault"
            if len(frames) < num_frames:
                frames.append(page)
            else:
                future = reference[i + 1:]
                # Tie rule required by the assignment: replace the first frame
                # whose page is never used again; otherwise farthest next use.
                next_positions = []
                for candidate in frames:
                    try:
                        next_positions.append(future.index(candidate))
                    except ValueError:
                        next_positions.append(float("inf"))
                max_distance = max(next_positions)
                victim_index = next_positions.index(max_distance)
                replaced = frames[victim_index]
                frames[victim_index] = page
        trace.append({"step": i + 1, "page": page, "frames": frames.copy(),
                      "status": status, "replaced": replaced})
    return faults, hits, trace
```

[1] ✓ 0s

```python
def parse_reference_string(raw: str) -> List[int]:
    """Accept comma-separated or whitespace-separated non-negative integer pages."""
    tokens = raw.replace(",", " ").split()
    if not tokens:
        raise ValueError("Enter at least one page number.")
    try:
        pages = [int(token) for token in tokens]
    except ValueError as exc:
        raise ValueError("Invalid page reference: use only integer page numbers separated by commas or spaces.") from exc
    if any(page < 0 for page in pages):
        raise ValueError("Page numbers must be non-negative integers.")
    return pages

def print_trace(name: str, result: Tuple[int, int, List[Dict[str, Any]]], total: int) -> None:
    faults, hits, trace = result
    print(f"\\n{name}")
    frame_count = max(len(row["frames"]) for row in trace)
    headers = ["Step", "Page"] + [f"Frame {i+1}" for i in range(frame_count)] + ["Result", "Replaced"]
    print(" | ".join(headers))
    print("-" * (len(" | ".join(headers)) + 12))
    for row in trace:
        frame_values = [str(x) for x in row["frames"]] + ["-"] * (frame_count - len(row["frames"]))
        print(" | ".join([str(row["step"]), str(row["page"])] + frame_values +
                         [row["status"], "-" if row["replaced"] is None else str(row["replaced"])]))
    print(f"Page faults: {faults}")
    print(f"Page hits:   {hits}")
    print(f"Hit ratio:   {hits / total:.2%}")
    print(f"Fault ratio: {faults / total:.2%}")

def compare_algorithms(reference: List[int], num_frames: int, show_traces: bool = True):
    results = {
        "FIFO": fifo(reference, num_frames),
        "LRU": lru(reference, num_frames),
        "Optimal": optimal(reference, num_frames),
    }
    if show_traces:
        for name, result in results.items():
            print_trace(name, result, len(reference))
    print("\\nPERFORMANCE COMPARISON")
    print(f"{'Algorithm':<12} {'Faults':>8} {'Hits':>8} {'Hit ratio':>12} {'Fault ratio':>12}")
    print("-" * 56)
```

```
    for name, (faults, hits, _) in results.items():
        print(f"{name:<12} {faults:>8} {hits:>8} {hits/len(reference):>11.2%} {faults/len(reference):>11.2%}")
    return results

def run_interactive():
    try:
        raw = input("Enter page reference string (comma or space separated): ")
        reference = parse_reference_string(raw)
        raw_frames = input("Enter number of frames: ").strip()
        try:
            num_frames = int(raw_frames)
        except ValueError as exc:
            raise ValueError("Number of frames must be a whole number.") from exc
        if num_frames < 1:
            raise ValueError("Number of frames must be at least 1.")
        compare_algorithms(reference, num_frames)
    except ValueError as error:
        print(f"Input error: {error}")
```

[2] ✓ 0s

```
sample_reference = [1, 2, 3, 1, 4, 2, 5]
sample_frames = 3
sample_results = compare_algorithms(sample_reference, sample_frames)
```

```
\nFIFO
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
--------------------------------------------------------------------------------
1 | 1 | 1 | - | - | Fault | -
2 | 2 | 1 | 2 | - | Fault | -
3 | 3 | 1 | 2 | 3 | Fault | -
4 | 1 | 1 | 2 | 3 | Hit | -
5 | 4 | 4 | 2 | 3 | Fault | 1
6 | 2 | 4 | 2 | 3 | Hit | -
7 | 5 | 4 | 5 | 3 | Fault | 2
Page faults: 5
Page hits:   2
Hit ratio:   28.57%
Fault ratio: 71.43%
\nLRU
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
--------------------------------------------------------------------------------
1 | 1 | 1 | - | - | Fault | -
2 | 2 | 1 | 2 | - | Fault | -
3 | 3 | 1 | 2 | 3 | Fault | -
4 | 1 | 1 | 2 | 3 | Hit | -
5 | 4 | 1 | 4 | 3 | Fault | 2
6 | 2 | 1 | 4 | 2 | Fault | 3
7 | 5 | 5 | 4 | 2 | Fault | 1
Page faults: 6
Page hits:   1
Hit ratio:   14.29%
Fault ratio: 85.71%
\nOptimal
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
--------------------------------------------------------------------------------
1 | 1 | 1 | - | - | Fault | -
2 | 2 | 1 | 2 | - | Fault | -
3 | 3 | 1 | 2 | 3 | Fault | -
4 | 1 | 1 | 2 | 3 | Hit | -
5 | 4 | 4 | 2 | 3 | Fault | 1
6 | 2 | 4 | 2 | 3 | Hit | -
7 | 5 | 5 | 2 | 3 | Fault | 4
Page faults: 5
Page hits:   2
Hit ratio:   28.57%
Fault ratio: 71.43%
\nPERFORMANCE COMPARISON
Algorithm      Faults     Hits    Hit ratio  Fault ratio
------------------------------------------------------------
FIFO                5        2      28.57%      71.43%
LRU                 6        1      14.29%      85.71%
Optimal             5        2      28.57%      71.43%
```

```python
experiment_reference = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
for frame_count in (3, 4):
    print(f'--- {frame_count} frames ---')
    results = compare_algorithms(experiment_reference, frame_count, show_traces=False)
```

```
--- 3 frames ---
\nPERFORMANCE COMPARISON
Algorithm      Faults     Hits    Hit ratio  Fault ratio
------------------------------------------------------------
FIFO                9        3      25.00%      75.00%
LRU                10        2      16.67%      83.33%
Optimal             7        5      41.67%      58.33%
--- 4 frames ---
\nPERFORMANCE COMPARISON
Algorithm      Faults     Hits    Hit ratio  Fault ratio
------------------------------------------------------------
FIFO               10        2      16.67%      83.33%
LRU                 8        4      33.33%      66.67%
Optimal             6        6      50.00%      50.00%
```

```python
lru_better_reference = [5, 4, 1, 5, 1, 2, 5, 1, 5, 5, 4, 1, 2, 1, 5, 2, 3, 4]
fifo_equal_reference = [5, 1, 5, 3, 5, 2, 1, 5, 5, 2, 3, 1, 5, 1, 5, 1, 5, 2]
print('LRU better:', len(lru_better_reference), 'references, 3 frames')
print('FIFO faults:', fifo(lru_better_reference, 3)[0], '| LRU faults:', lru(lru_better_reference, 3)[0])
print('FIFO better/equal:', len(fifo_equal_reference), 'references, 4 frames')
print('FIFO faults:', fifo(fifo_equal_reference, 4)[0], '| LRU faults:', lru(fifo_equal_reference, 4)[0])
```

```
LRU better: 18 references, 3 frames
FIFO faults: 11 | LRU faults: 9
FIFO better/equal: 18 references, 4 frames
FIFO faults: 4 | LRU faults: 4
```

0s

```python
manual_reference = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
manual_results = compare_algorithms(manual_reference, 3)
```

```
\nFIFO
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
------------------------------------------------------------------------------
1 | 7 | 7 | - | - | Fault | -
2 | 0 | 7 | 0 | - | Fault | -
3 | 1 | 7 | 0 | 1 | Fault | -
4 | 2 | 2 | 0 | 1 | Fault | 7
5 | 0 | 2 | 0 | 1 | Hit | -
6 | 3 | 2 | 3 | 1 | Fault | 0
7 | 0 | 2 | 3 | 0 | Fault | 1
8 | 4 | 4 | 3 | 0 | Fault | 2
9 | 2 | 4 | 2 | 0 | Fault | 3
10 | 3 | 4 | 2 | 3 | Fault | 0
11 | 0 | 0 | 2 | 3 | Fault | 4
12 | 3 | 0 | 2 | 3 | Hit | -
13 | 2 | 0 | 2 | 3 | Hit | -
Page faults: 10
Page hits:   3
Hit ratio:   23.08%
Fault ratio: 76.92%
```

```
\nLRU
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
--------------------------------------------------------------------------------
1 | 7 | 7 | - | - | Fault | -
2 | 0 | 7 | 0 | - | Fault | -
3 | 1 | 7 | 0 | 1 | Fault | -
4 | 2 | 2 | 0 | 1 | Fault | 7
5 | 0 | 2 | 0 | 1 | Hit | -
6 | 3 | 2 | 0 | 3 | Fault | 1
7 | 0 | 2 | 0 | 3 | Hit | -
8 | 4 | 4 | 0 | 3 | Fault | 2
9 | 2 | 4 | 0 | 2 | Fault | 3
10 | 3 | 4 | 3 | 2 | Fault | 0
11 | 0 | 0 | 3 | 2 | Fault | 4
12 | 3 | 0 | 3 | 2 | Hit | -
13 | 2 | 0 | 3 | 2 | Hit | -
Page faults: 9
Page hits:   4
Hit ratio:   30.77%
Fault ratio: 69.23%
\nOptimal
Step | Page | Frame 1 | Frame 2 | Frame 3 | Result | Replaced
--------------------------------------------------------------------------------
1 | 7 | 7 | - | - | Fault | -
2 | 0 | 7 | 0 | - | Fault | -
3 | 1 | 7 | 0 | 1 | Fault | -
4 | 2 | 2 | 0 | 1 | Fault | 7
5 | 0 | 2 | 0 | 1 | Hit | -
6 | 3 | 2 | 0 | 3 | Fault | 1
7 | 0 | 2 | 0 | 3 | Hit | -
8 | 4 | 2 | 4 | 3 | Fault | 0
9 | 2 | 2 | 4 | 3 | Hit | -
10 | 3 | 2 | 4 | 3 | Hit | -
11 | 0 | 2 | 0 | 3 | Fault | 4
12 | 3 | 2 | 0 | 3 | Hit | -
13 | 2 | 2 | 0 | 3 | Hit | -
Page faults: 7
Page hits:   6
Hit ratio:   46.15%
Fault ratio: 53.85%
```

```
\nPERFORMANCE COMPARISON
Algorithm      Faults     Hits    Hit ratio  Fault ratio
------------------------------------------------------------
FIFO               10        3      23.08%      76.92%
LRU                 9        4      30.77%      69.23%
Optimal             7        6      46.15%      53.85%
```